
Design layer

the whole layer, as one document

Contents

1 Saga	4
1.1 Foundation	4
1.2 Pending updates	5
1.3 System at a glance	6
1.4 Workflow model	7
1.5 Typed authoring	8
1.5.1 Native transformations and diagnostics	9
1.6 Coordinator internals	10
1.6.1 Value storage and soundness	10
1.6.2 Readiness and worker lifetime	11
1.7 Execution lifecycle	11
1.8 Step action lifecycle	12
1.9 Compensation and uncertainty	14
1.10 Outcome and settlement	15
1.11 Time and capacity	16
1.12 Local ownership and reporting	17
1.12.1 One invocation receiver	18
1.13 Durable execution model	18
1.14 Checkpoint and effect permission	20
1.15 Recovery rules	21
1.16 Storage and claim lifetime	22
1.17 Storage adapters and delivery	24
1.18 Observation and verification ports	25
1.19 Retained composition contracts	26
1.19.1 Value-dependent traversal and local continuation	26
1.19.2 Approvals and historical review	26
1.19.3 Independent children and compensation authority	27
1.20 Runtime-authored graphs	28
1.20.1 Parametric node model	28
1.20.2 Publication and hostile input	29
1.21 End-to-end walkthrough	30
Glossary	30

ROOT DESIGN

Saga

1 Saga

1.1 Foundation

Saga owns typed dependency execution and the evidence needed to settle effects.

GOAL

Execute native typed workflows with complete evidence

One definition supports shared dependencies, concurrency, retry, compensation, and undo while retaining caller-owned values and failure types.

GOAL

Recover saved execution without guessing effect absence

Optional persistence retains progress and stable identities across runner or VM loss. Recovery uses explicit application evidence before repeating admitted work.

NO-GOAL

A job-delivery or agent-policy runtime

Delivery, queue capacity, agent policy, protocol transport, and application retry policy belong to their respective owners.

NO-GOAL

Serialize closures or expose arbitrary native lookup

Executable closures are reconstructed from deployed definitions. Native dependency values cannot be obtained through runtime result names or caller-visible type erasure.

INVARIANT

A native dependency preserves its producer type

An opaque Port binds one produced value type to every consumer. Only construction of that Port can establish its node identity and typed read capability.

mechanism

INVARIANT

An uncertain action remains recorded after recovery

A later retry, replacement, abort, or hold does not erase an earlier unknown effect. Completion with such evidence has a distinct outcome.

mechanism

INVARIANT

Only known completions enter the undo journal

An interrupted attempt is never reported as undone. Successful undo entries run in reverse actual completion order and retain every failure.

mechanism

INVARIANT

Durable effect permission follows a committed state

Admission and checked input are saved before a worker receives permission for the external action. A refused commit stops further dispatch.

mechanism

INVARIANT

A current claim guards every checkpoint write

The adapter checks claim identity, cancellation observation, and revision atomically. A released or replaced claim cannot write progress.

mechanism

PRINCIPLE

Separate evidence from authority

An observed result, cancellation request, notification, or telemetry event cannot establish authority it does not carry. Compensation authority and absence of external effects require explicit contracts.

PRINCIPLE

Preserve local capability without persistence ceremony

Native local execution requires no codecs or storage. Persistence attaches to the same definition and checks its additional obligations.

PRINCIPLE

Keep package boundaries at distinct lifecycles

Saga owns effect progress and settlement. Storage owns persistent ownership, while applications compose delivery and agent lifetimes through small public ports.

1.2 Pending updates

build Value-dependent composition and bounded traversal

Add typed value-dependent composition and homogeneous traversal with ordering, graph-size, cancellation, and nested-capacity contracts. Preserve arbitrary local typed fragments; their exact admission and durable eligibility need a ruling before an API is accepted.

ADR-0008

ruling Approval and historical fork authority

Retain durable approval waits, authenticated revision-checked deduplicated signals, and historical review forks. The owner must decide how a changed answer treats prior effects and which continuation owner suspends a Fabric agent.

ADR-0008

build Independent children and completed-run compensation

Implement typed admission, attachment, observation, command reconciliation, journal retention, and exclusive compensation authority. Scripted laboratory records establish intended distinctions but do not establish atomic admission, authorization, races, or recovery.

ADR-0008

build Runtime graph publication and execution

Implement finite-contract registered graphs, parametric node instantiation, exact connection compatibility, publication checks, and execution. Generic node families, native adapters, LLM nodes, visualization, and persistence remain required within this surface.

ADR-0008

build Extended graph validation and introspection

Add declared associative multiple-writer reducers, separate diagnostic labels and metadata, and graph visualization export. Preserve pure authoring without OTP startup; static construction already excludes cycles and missing producers.

ADR-0008

ruling Shared capacity and explicit lifecycle extensions

Decide shared or partitioned budgets across runs and children, coordinator supervision, explicit in-memory halt/resume, and undo retry policy. Current concurrency is per run and each undo is attempted once.

ADR-0008

ruling Checkpoint migration and delivery transaction

Define explicit definition/checkpoint migration and any optional Saga/Grind runner or outbox. An application currently owns wakeups, transaction boundaries, and redelivery; no generic durable runtime is inferred from similar claims or durations.

ADR-0004

ruling A total drive return deadline needs an owner decision

The drive execution budget is followed by synchronous runner drain and possible bounded claim release. A stronger total-return promise requires an explicit cleanup owner and takeover contract.

ADR-0009

ruling The common example must retain typed failures

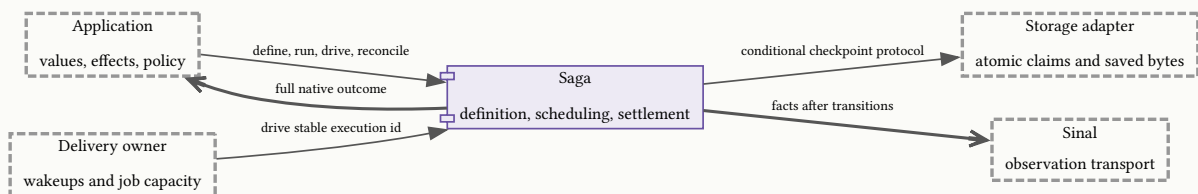
The README and execution module example reduce all noncompletion to unknown-effects lists and admission failure to an empty list. Correcting the executable example and its consumer assertion requires a coordinated example change; the library already returns full typed reports.

ADR-0009

1.3 System at a glance

One workflow model is interpreted by the local coordinator or by a checkpointed drive.

VIEWPOINT RUNTIME · ALTITUDE L1 · BOUNDARY Ownership around Saga



LEGEND



Storage implements a port; it does not choose workflow progress. Delivery acknowledgments and Sinal events never replace the saved outcome.

- The application supplies native input/output/error/undo-error records and callbacks. Saga retains them through **Port** connections and **Outcome** reports.
- A **Local run** belongs to its starting process; owner exit requests cancellation. A **Durable execution** belongs to its saved record; driver exit stops driving without recording cancellation.
- The package requires Sinal and standard Gleam/OTP/time/JSON libraries. PostgreSQL, Blueprint, Grind, Fabric, Relay, and provider clients compose through optional adapters or application code.

Saga

responsibility

Execute typed dependencies and preserve effect evidence.

interface

Workflow authoring; local execution; optional durable capability; storage, outcome, reporting, observation, and testing ports.

interactions

Calls application effects and recovery callbacks; interprets storage conditional writes; emits observations through Sinal.

invariants

One native type relation per Port; bounded per-run admission; uncertainty and settlement survive projection boundaries.

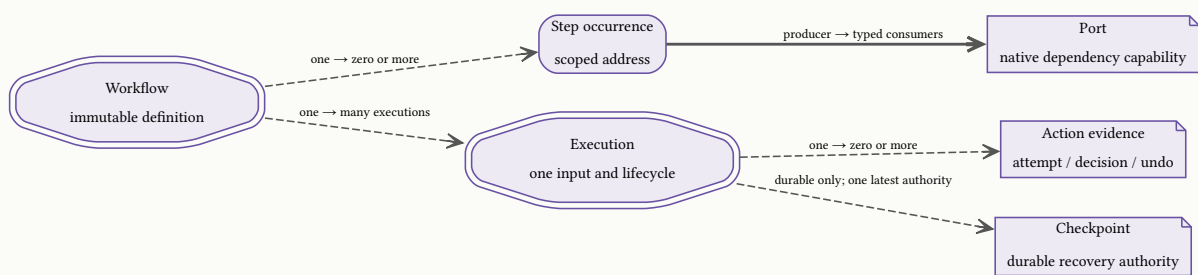
failure

Separate definition/configuration rejection, workflow outcomes, durable operational failures, and lost report delivery.

1.4 Workflow model

Definitions, execution identity, and observed action results are separate planes.

VIEWPOINT DOMAIN-MODEL · ALTITUDE L2 · CONTEXTS The model and its cardinalities



LEGEND



A step definition can be used in multiple occurrences. Runtime node numbers support scheduling but do not become stable persistence identities.

Workflow aggregate immutable workflow owned by Saga authoring

The validated reusable native dependency definition.

ATTRIBUTES

authored **Native contract** · Input × output × business-error × undo-error
Caller types remain generic throughout construction and execution.

authored **Definition name** · Diagnostic workflow name
Names the definition; it does not identify one execution.

derived **Step occurrences and dependencies** · Validated acyclic typed dependency structure
Derived once when the builder returns its output Port.

RELATIONSHIPS

1 : 0..n Defines step occurrences and supports separate executions.

Step and Port

value-object

immutable

workflow

owned by Saga authoring

The operation contract and its construction-owned native dependency capability.

ATTRIBUTES

authored

Step policy · Attempt × recovery × undo × timing × persistence capability
A Step can be configured without running its callback. Persistence is optional.

derived

Port membership · Construction identity and typed fetch
A Port can only be read at the type paired with its producer. Foreign membership is rejected.

RELATIONSHIPS

1 : 0..n

One produced Port may feed several consumers, sharing one scheduled producer.

Effect key

value-object

immutable

workflow

owned by Saga execution

The opaque context of one attempt, compensation decision, or undo.

ATTRIBUTES

derived

Idempotency identity · Execution × step address × action family
Stable across attempts of the same step and across durable restart. An undo has its own identity.

derived

Attempt identity · Action identity × positive attempt number
Distinguishes attempt evidence; its text is opaque and must never be parsed.

derived

Correlation · Sinal Correlation
Chosen at local start or saved at the first durable drive.

Value or sum	Meaning	Refinement
StepAddress	Scope, name, occurrence	Occurrence is positive; scope records embed/choice structure.
AttemptFailure	Returned(error), Crashed(crash), TimedOut	Closed; preserves native error independently from effect status.
Undo	NoUndo or application action	NoUndo is absence, never a successful no-op.
Timeout	After(Duration) or Infinity	Positive millisecond resolution where required; explicit Infinity lifts the bound.
Recovery	Retry, RetryAfter, Continue, Abort, AbortAfterCleanupFailure, Hold	Closed; replacement includes its own undo capability.
Outcome	Completed, CompletedWithUnknownEffects, Failed, Cancelled, Unresolved	Evidence-bearing alternatives have different payloads.

1.5 Typed authoring

Construction establishes dependencies once; execution reuses the validated graph.

Definition builder

Bind typed dependencies and validate every constructed step.

responsibility

Evaluate the builder once and collect every constructed occurrence.

interface

define panics for source defects; try_define returns all DefinitionErrors; name and describe expose read-only metadata.

Typed composition

Compose values while retaining one run and undo journal.

responsibility

Connect perform, both, nonempty all, map, embed, and closed choose.

interface

Opaque Port and Workflow values; caller-owned error adaptation through map_step_errors and map_errors.

interactions

Mints construction membership and Ports; collects reachable steps from the output; checks unused constructed steps.

invariants

Empty names, invalid budgets/timeouts, foreign Ports, and orphan steps are rejected together. Construction invokes no step effects.

failure

Builder raises become a located definition failure; `define` reports the workflow and offending steps. Runtime-derived settings use `try_define`.

interactions

Each embed introduces a distinct scope; each choose builds both branches and schedules a decision. Shared dependencies remain shared.

invariants

Closed branches have the same output/error types; only the selected branch executes. `map_errors` reuses the graph and never reevaluates the builder.

failure

Compiler rejects mismatched outputs/errors; runtime definition validation rejects foreign/orphan construction. Output transformation crashes become `OutputCrashed`.

1.5.1 Native transformations and diagnostics

- `map` transforms a Port's fetched value; it is not a scheduled memoized step. It may execute in each consumer task; expensive or effectful work uses a Step.
- `all` requires a first Port and accepts further Ports of the same native type. The caller handles an empty collection explicitly.
- `StepDescriptor` contains address, dependency addresses, undo capability, compensation capability, attempt limit, and optional step timeout. It exposes neither callbacks nor executable control over child internals.
- Step names are diagnostic and structural identity, not native value lookup. Reassociation or different scopes can change durable identity even when the final business value remains equivalent.

BEHAVIOR

A shared producer runs once

Given Several consumers depend on the same Port in one execution.

When The workflow executes.

Then The producer succeeds at most once under its retry policy, and every consumer reads that committed output.

level: boundary area: Authoring

BEHAVIOR

Unselected branch effects do not execute

Given The workflow contains a closed choice.

When The choice decision selects one branch.

Then Only that branch's effects execute; shared prerequisites remain shared.

Then Durable recovery preserves the selection before further branch effects.

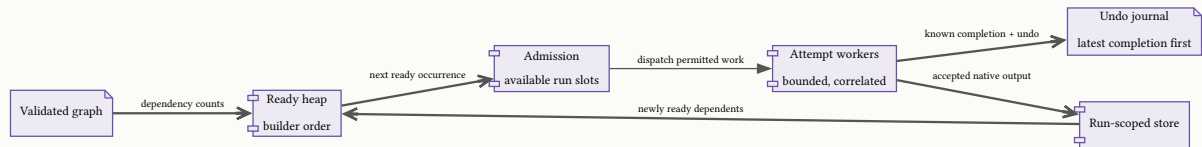
level: boundary area: Closed choice

- Authoring and scope semantics are owned here; the representation decision is `ADR-0001`. The build-once storage trade is `ADR-0002`.

1.6 Coordinator internals

Run state owns values, readiness, workers, and the completion journal together.

VIEWPOINT DATAFLOW · ALTITUDE L4 · INTERNALS A coordinator advances one run



LEGEND


value object
attribute-defined value


component
owned runtime part


dataflow
weighted directed link


call
solid directed link

A durable commit gates dispatch across these transitions. The coordinator receives uniform action events while typed node closures own native values.

Coordinator

responsibility

Serialize transitions of one run and enforce readiness, capacity, timers, settlement, and rollback.

interface

Uniform completion/control messages; read-only progress; one terminal delivery function.

interactions

Interprets immutable node closures with a fresh store, monitors owner/workers, and optionally commits checkpoint values.

invariants

Only ready work enters a slot; attempts and compensation decisions share max_concurrency; retries reenter admission instead of bypassing capacity.

failure

A terminal trigger stops admission and initiates settlement. Durable storage/refusal or driver loss stops dispatch under the last checkpoint authority.

1.6.1 Value storage and soundness

Run-scoped store

responsibility

Retain completed native outputs and typed action records for one run.

interface

Internal opaque Store; node-bound put/fetch operations.

interactions

The Port's typed fetch and producer are created together; the coordinator threads a fresh store through run state.

invariants

No other run shares the store. Only the producer-bound native type reads a node's value; type erasure is confined to this module.

failure

Reading a value before dependency commit is an internal defect and panics. Opaque Port construction and compiler-negative fixtures exclude foreign typed reads.

- The implementation uses an opaque native carrier and an identity cast, never caller-visible Dynamic. Readiness and one binding of node identity to native type make that cast sound; [ADR-0002](#) records the choice.

1.6.2 Readiness and worker lifetime

Readiness index

responsibility

Admit ready nodes without rescanning the whole graph.

interface

Dependency counts, dependents, ordered ready min-heap, active count, completed count.

interactions

A committed success lowers dependent counts; retry readiness returns to the heap.

invariants

No dependent fetch precedes committed output. Builder order breaks readiness ties, while actual completion order defines undo order.

failure

Waiting nodes become skipped after admission closes; stale timer/worker messages cannot advance another action sequence.

Action workers

responsibility

Isolate attempt, compensation, and undo execution from coordinator transitions.

interface

Correlated action sequence, result, monitor, timeout, and durable permission channel.

interactions

Attempts prepare typed input; persistence checks that input before effect permission; workers report returned result or typed crash evidence.

invariants

A stale completion cannot settle a newer action. A worker killed before permission cannot start its effect; a killed admitted action remains uncertain.

failure

Raises and exits become Crash; time limits kill workers and retain unknown-effect evidence. Detached durable runners stop their workers without inventing cancellation.

1.7 Execution lifecycle

Admission, sibling settlement, and reverse undo are distinct phases.

Execution phase	running	Admits ready work.	settling	Waits for active siblings without admitting new work.
	rolling-back	Runs known undo entries sequentially.	finished	Retains one terminal outcome; coordinator exits.

Execution aggregate stateful workflow owned by Saga coordinator

One input interpretation of a Workflow and its effect evidence.

ATTRIBUTES

- derived** **Run identity** · Local fresh run id or durable execution id
Local id is fresh per start; the durable id is caller-chosen within one store.
- derived** **Phase** · Execution phase [execution-phase](#) · [execution-lifecycle](#)
Derived from accepted transitions; public progress exposes the three active phases only.
- derived** **Primary trigger** · Failure, cancellation, or held evidence
The original stopping cause remains separate from subsequently observed sibling and cleanup failures.

observed

Action evidence · Known results, journal, and unknown actions
Accepted callback results record observations; recovery decisions supply explicit transition authority.

RELATIONSHIPS

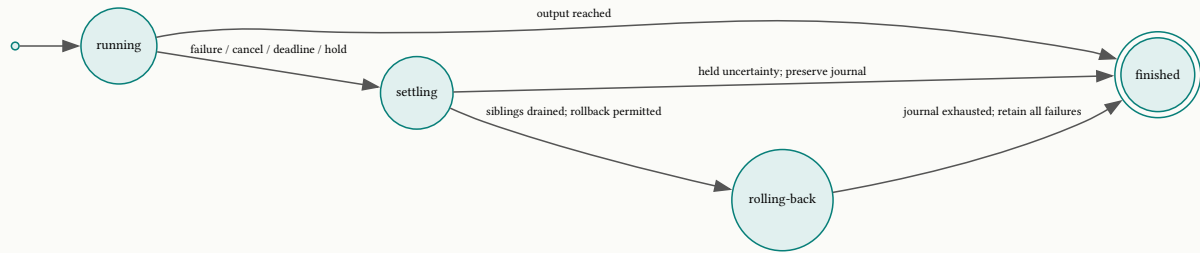
1 : 1

Interprets one Workflow definition.

1 : 0..n

Owns attempts, compensation decisions, and journal entries.

STATE MACHINE The execution phases



Cancellation after settlement has begun is idempotent and does not replace the stopping trigger. A completed last step can still be undone when cancellation wins before terminal completion.

From	Event and guard	Next state	Evidence or refusal
Running	Known output reached before a stopping trigger	Finished	Completed, or CompletedWithUnknownEffects.
Running	Failure, explicit cancel, local owner exit, deadline, or Hold	Settling	Skip waiting steps; preserve primary trigger.
Settling	Active work returns before settle bound	Settling	Accept known completions and retain later failures.
Settling	Settle bound expires	RollingBack or Finished	Kill remaining work and record interruption; Hold forbids rollback.
RollingBack	An undo returns, fails, crashes, or times out	RollingBack	Continue remaining entries; retain every located result.
Settling / RollingBack / Finished	Repeated cancellation	Unchanged	No new admission and no replacement of the original trigger.

1.8 Step action lifecycle

A retry creates another attempt while preserving prior action evidence.

Step

progress

waiting attempting compensating retry-scheduled succeeded failed-step interrupted undoing undone
undo-failed-step skipped

Step occurrence

entity stateful workflow owned by Saga coordinator

A structurally addressed operation within one Execution.

ATTRIBUTES

derived

Address · Step address
Resolved from construction scope, name, and occurrence.

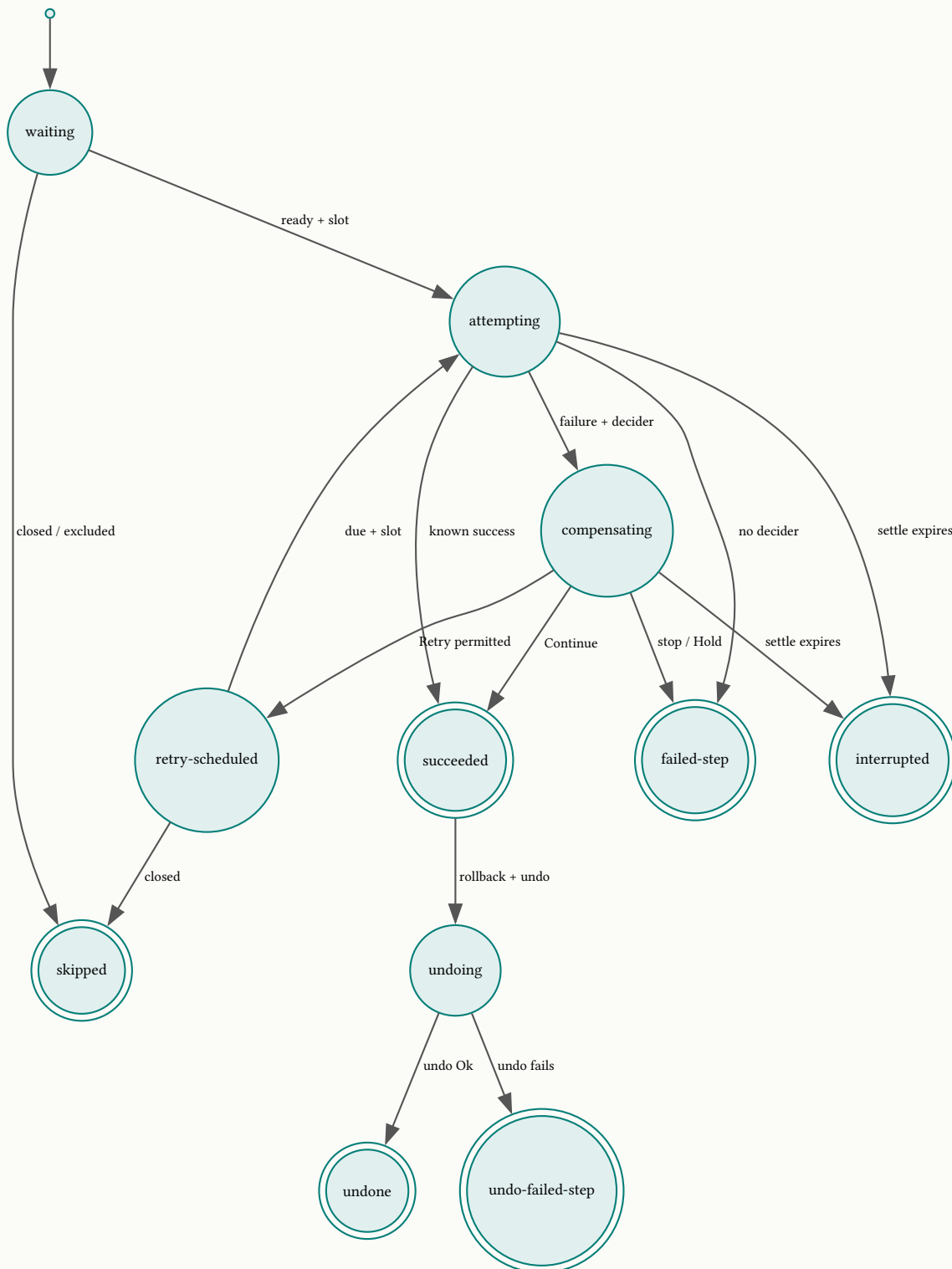
derived

State · Step progress [step-progress](#) · [step-lifecycle](#)
Attempt numbers and correlated action sequences are associated data.

derived

Attempts remaining · Finite positive budget including the first attempt
A compensation decision still runs after the last allowed attempt because cleanup may be needed.

STATE MACHINE Step progress and rollback



Succeeded remains terminal when no rollback is needed or no undo exists. FailedStep includes the stopped node of a held run; it does not imply that the effect is known.

- Public Progress reports phase and every StepProgress without native values or closures. It is a read-only observation, never a control or recovery token.

1.9 Compensation and uncertainty

Failed-attempt decisions and completed-step undo grant different permissions.

Recovery policy

responsibility

Choose how a failed attempt affects its own progress and prior completed work.

interface

FailedAttempt(input, failure, attempt, attempts_left, key) → Recovery(output, error, undo_error).

interactions

Receives returned native errors and native crash/timeout evidence; may perform cleanup under its own bound.

invariants

Retry does not erase uncertainty; Continue supplies the replacement output's undo; Hold grants neither retry nor rollback authority.

failure

A raising or timed-out decider retains unknown compensation evidence; AbortAfterCleanupFailure retains its cleanup error separately.

Decision	Permitted transition	Required retained evidence
Retry / RetryAfter	Another attempt only within budget and open admission; delay capped	Earlier unknown action stays recorded; retry-superseded differs from exhausted budget.
Continue(output, undo)	Accept replacement output and its undo capability	Replacement completion is known; original uncertain attempt remains unknown.
Abort(error)	Fail and permit reverse undo of known completions	Native application cause plus earlier unknown effects.
AbortAfterCleanupFailure(error, undo_error)	Same rollback permission	Separate compensation cleanup failure.
Hold(evidence)	End Unresolved after siblings settle; do not undo	Native held evidence and every completed step left in place.

- unknown_when marks returned business errors whose external effect may have happened. Without a deciding compensation, default on_unknown(Reconcile) preserves completed reservations; explicit RollBack permits their undo.
- Exhausted retry of a marked uncertain error follows the same Hold/RollBack policy. An explicit Abort permits rollback even after a marked error.
- Crashes and timeouts without a decider fail and roll back known completions while retaining the crashed/timed-out action as unknown. A decider's Abort may produce StepFailed after a crash; the cause alone cannot establish known effects.
- Applications own downstream idempotency and reconciliation evidence. Passing an idempotency key alone never proves provider deduplication or exactly-once effects.

BEHAVIOR

An uncertain retry cannot become clean completion

Given An attempt ended with unknown effect evidence.

When A later attempt or replacement reaches the workflow output.

Then The outcome retains the output and the earlier unknown effect in CompletedWithUnknownEffects.

level: boundary area: Unknown effects

BEHAVIOR

Held uncertainty preserves prior completions

Given A step returns uncertain evidence without rollback permission.

When The run settles.

Then The outcome is Unresolved and completed steps remain held.

Then No earlier completion is claimed as reversed.

level: boundary area: Compensation

- The uncertainty and rollback decisions are recorded in `ADR-0003`.

1.10 Outcome and settlement

One full typed report is the authority for application and operational evidence.

Outcome	Payload	Meaning
Completed	Native output	Output reached with no unknown actions.
CompletedWithUnknownEffects	Native output + nonempty unknown effects	Output reached; earlier effects still require reconciliation.
Failed	Cause(error) + Settlement(error, undo_error)	Failure stops admission and settlement records rollback evidence.
Cancelled	CancelRequested or OwnerExited + Settlement	Cancellation intent stopped admission; it does not assert complete reversal.
Unresolved	Step address + native evidence + Settlement	Held uncertainty stopped progress without rollback authority.

Settlement

value-object

immutable

workflow

owned by Saga execution

The immutable evidence accompanying a stopped Outcome.

ATTRIBUTES

observed

Reversed and retained completions · Undone × not-undoable × held step addresses
Known undo success is distinct from a completed effect lacking undo or held without permission.

observed

Failed cleanup · All located undo and compensation failures
Native undo errors, crashes, and timeouts remain distinct; one failure never stops the remaining journal.

observed

Concurrent stopping evidence · Interrupted steps × sibling causes
The primary trigger remains separate from failures arriving during settlement.

observed

Unknown effects · Step × action × ending
Attempt and compensation number remain visible; identical addresses do not erase distinct actions.

Outcome projection

responsibility

Classify full reports conservatively and provide payload-safe summaries.

interface

`outcome.classify(report, explain), kind, held_steps, failure_kind, describe_failure, and summary.`

interactions

A single private evidence projection supplies classification and rendering; the caller retains the original Outcome.

invariants

A definite stopped result requires every effect known and none left in place. Summary excludes output, business/undo error payloads, and crash reasons.

failure

Completion with uncertainty, Hold, retained effects, failed undo, or unknown actions classify unresolved. Neither definite classification nor reporting NotStarted chooses retry policy.

Projection	Permitted result	Disclosure
Clean Completed	Native output	No payload-safe summary promise applies to the returned output itself.
Fully reversed typed failure or cancellation	Definitely(message)	Caller explain renders selected business error.
CompletedWithUnknownEffects or retained/unknown effects	Unknown(evidence)	Complete safe settlement summary; Unresolved adds explicit held-error explanation.
summary	Outcome/cause kinds and located evidence	Never invokes explain; distinct actions/attempts and all settlement categories remain.

- A safe summary keeps addresses visible. Typed cause accessors and caller-provided renderers may expose private application data; the application owns disclosure. [ADR-0007](#) records why reporting retains full reports before projection.

1.11 Time and capacity

A duration bounds its named operation; it is not an end-to-end return promise.

Bound	Default	Ownership and limit
Concurrent attempts + compensation decisions	Online scheduler count	Per run; at least one; retries need slots.
Run deadline	Infinity	After(Duration) is opt-in; triggers settlement and rollback.
Attempt timeout	60 seconds	Step-specific timeout overrides default in either direction; Infinity opts out for undeclared steps.
Sibling settlement	5 seconds	Can be zero; no new admission; active siblings may finish.
Each compensation / undo	5 seconds	Positive; undo runs sequentially and is not retried.
RetryAfter cap	5 minutes	Nonnegative; requested larger delay clamps and is observed.
Storage operation	5 seconds	with_call_timeout; minimum one millisecond; slow call suspends/stops runner.
Checkpoint bytes	16 MiB	Whole saved snapshot; oversize returns CheckpointTooLarge.
Receiver readiness	5 seconds	Proven prelaunch failure returns typed reporting error.
Reporting rollback / notification	Caller Duration	1 through $2^{32}-1$ milliseconds; each callback has this bound.
Drive execution	Required caller Duration	Positive; after expiry drain/release work precedes return.

$$B = \sum_s A_s T_s + \sum_s A_s C + \sum_s A_s R + S + JC$$

NOTATION

A_s	allowed attempts for step s
T_s	effective finite attempt timeout
C	cleanup timeout
R	retry-delay cap
S	settle timeout
J	known journal entries requiring undo

The sum overcounts parallel work and the final retry delay. It bounds the configured action waits, not arbitrary synchronous observer or extension latency.

- A finite deadline substitutes its execution portion for the summed action portion, then settlement and cleanup still occur. Infinity, caller callbacks without an applicable timer, or synchronous Sinal handlers can defeat a literal wall-clock bound.
- Local progress/await bounds limit the caller's observation wait; a timeout does not cancel the run. The caller explicitly cancels and consumes the terminal report before dropping an ordinary Execution handle.
- The current local start path contains an internal readiness wait and a separate public control-handshake wait, each five seconds. The module's single five-second wording is narrower than these sequential waits; no total-start bound is asserted here.
- A drive begins its execution-budget clock after spawning and monitoring the runner. Its drain receives wait up to five seconds each and reset after a claim/result message; the normal finite protocol has at most one claim and one result. An abnormal exit with a known claim then performs one separately bounded release call.
- Thus drive may return after its requested Duration. Drain scheduling and release timing are explicit margin obligations when an application nests drive under a job worker deadline; a stronger total deadline remains a pending ruling.
- Persistent deadlines/backoff use wall-clock timestamps across restart; current run timers use monotonic timing. The deployment owns wall-clock suitability and cannot infer cross-node agreement from local Duration types.

BEHAVIOR

Cancellation settles known concurrent work

Given A local run has active siblings.

When Explicit cancellation or owner exit stops admission.

Then Active siblings may finish within the configured settle window.

Then Remaining actions are interrupted and unknown; only known completed effects enter rollback.

level: boundary area: Local lifetime

1.12 Local ownership and reporting

The starting process owns execution, while outcome delivery may belong to a survivor.

Local execution port

responsibility

Start one owner-bound coordinator and expose its outcome, cancellation, and progress.

interface

run, start, await, start_reporting, cancel, progress, pid, and run_id.

interactions

Monitors starting process; ordinary handle delivers to that owner, reporting handle delivers once to its Subject.

invariants

Only the ordinary starter may await. Reporting handles cannot await; cancel/progress work from any process.

failure

InvalidConfig starts no work; ExecutionLost cannot prove no effect started. Await distinguishes timeout, NotOwner, consumed outcome, and typed Lost evidence.

- A successful ordinary await consumes the outcome and drains its monitor. A second await may first time out while the coordinator is exiting; AlreadyAwaited is guaranteed only after exit.
- A dropped handle after a timed-out await can leave its terminal outcome or monitor message in the owner mailbox. Cancellation requires a subsequent await to consume completion.
- `start_reporting` allocates no outcome monitor/mail in the starting process. Its Subject receives at most one terminal report after settlement; coordinator death without a report remains distinguishable through monitoring.

1.12.1 One invocation receiver

Owned reporting

responsibility

Preserve a full typed report across abnormal exit of one invocation owner.

interface

`reporting.run_owned(workflow, input, config, on_stopped, rollback_within)` returns `Result(Outcome, reporting.Error)`; callback receives the same result.

interactions

Starts an independent receiver and waits for readiness before execution; receiver monitors invocation and coordinator; a guarded worker invokes `on_stopped`.

invariants

Receiver lives until invocation exit even after synchronous return. Normal owner exit produces no second notification; abnormal exit can deliver settlement.

failure

Startup exit/timeout, admission, receiver loss, and coordinator loss retain typed operational causes; callback crash/timeout leaves notification unconfirmed.

- Use this boundary within a per-invocation worker. Repeated use from a long-lived actor retains one receiver per invocation until that actor exits.
- `rollback_within` bounds owner-loss waiting when no coordinator was reported and bounds the notification worker. Once a known coordinator exists, its configured settlement/cleanup govern completion; `rollback_within` is not a global execution deadline.
- Readiness failure closes the startup reply channel, removes monitors, and preserves unrelated mail before any workflow launch. Normal result return and admission failure remove the caller's receiver monitor.
- `NotStarted` is proven only for rejected rollback/configuration or prelaunch receiver failure. Lost execution handshake, receiver loss, and coordinator loss are `Unknown`; typed `run_error/exit_reason` accessors retain causes and `safe_describe_error` excludes crash payloads.

1.13 Durable execution model

A saved execution outlives a drive; a fresh compatible definition supplies executable behavior.

Saved status

`pending` `suspended` `finished`

Durable execution

`aggregate` `stateful` `workflow` `owned by Saga durable protocol`

The storage-addressed aggregate that retains input, progress, compatibility, and outcome.

ATTRIBUTES

`authored`

Execution identifier · Caller-chosen id within `Storage`
A stable application key normally includes workflow identity. It is independent from local runner and correlation identity.

derived **Compatibility and input** · Computed stamp × checked encoded native input
Reconnect requires compatible definition and exactly the same encoded input.

derived **Status** · Saved status [saved-status](#) · [saved-lifecycle](#)
One latest committed authority; a suspended reason retains typed failure details.

authored **Cancellation intent** · Monotonic requested flag
Independent from status and claim; a terminal commit must observe its current value.

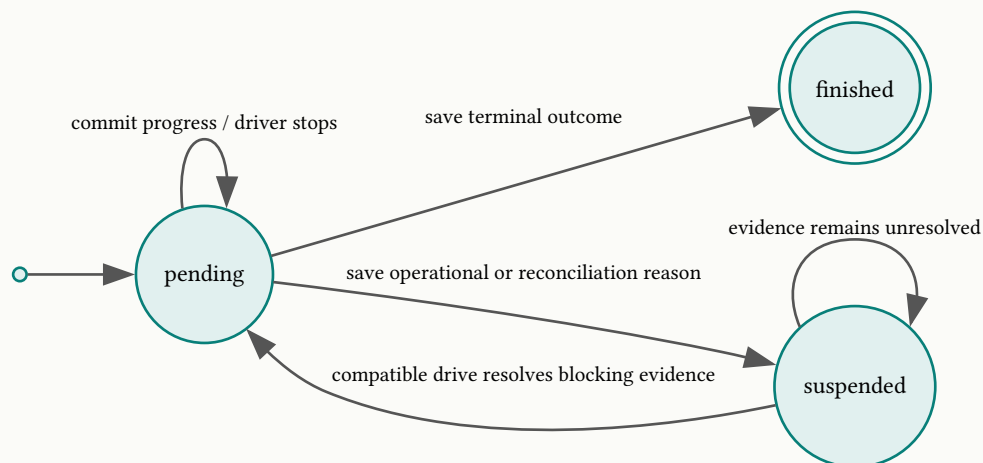
derived **Checkpoint revision** · Increasing Revision
Changes only on accepted checkpoint commits, not cancellation alone.

derived **Persisted correlation** · Absent before first drive, then one Sinal Correlation
The first drive chooses and commits it; every later handle uses the saved value.

RELATIONSHIPS

1 : 0..1 Has one current Claim; lack of a runner does not make it cancelled.

STATE MACHINE The saved execution lifecycle



Cancellation is an independent intent axis. Driver timeout, death, or lost ownership leaves the latest saved state authoritative and does not introduce cancellation.

Persistence capability

responsibility

Check an existing Workflow's eligibility and restore-compatible identity.

interface

durable.new with labeled root codecs; with_version, with_config, with_max_checkpoint_bytes; step recoverable/restore_undo/resolvers.

interactions

Uses step versions, graph wiring, and codec identities to compute the stamp; attaches no second graph.

invariants

Every persistent step is recoverable; every compensating step declares pure undo reconstruction including explicit NoUndo. Compatibility precedes application decoding.

failure

Missing/empty persistent capabilities are source defects and new panics with all problems. Changed stamps return IncompatibleDefinition before decoder callbacks.

Durable handle and driver

responsibility

Create/reconnect one saved execution and advance it under a current claim.

interface

start_or_reconnect, reconnect, read, cancel, drive, unfinished; growing Error with stable error_kind.

interactions

One Storage serves all ids; drive claims, restores, runs, saves outcome, stops helpers, and releases.

invariants

Create/reconnect runs no effects; same id/definition/encoded input is idempotent. Finished executions return saved Outcome without another effect.

failure

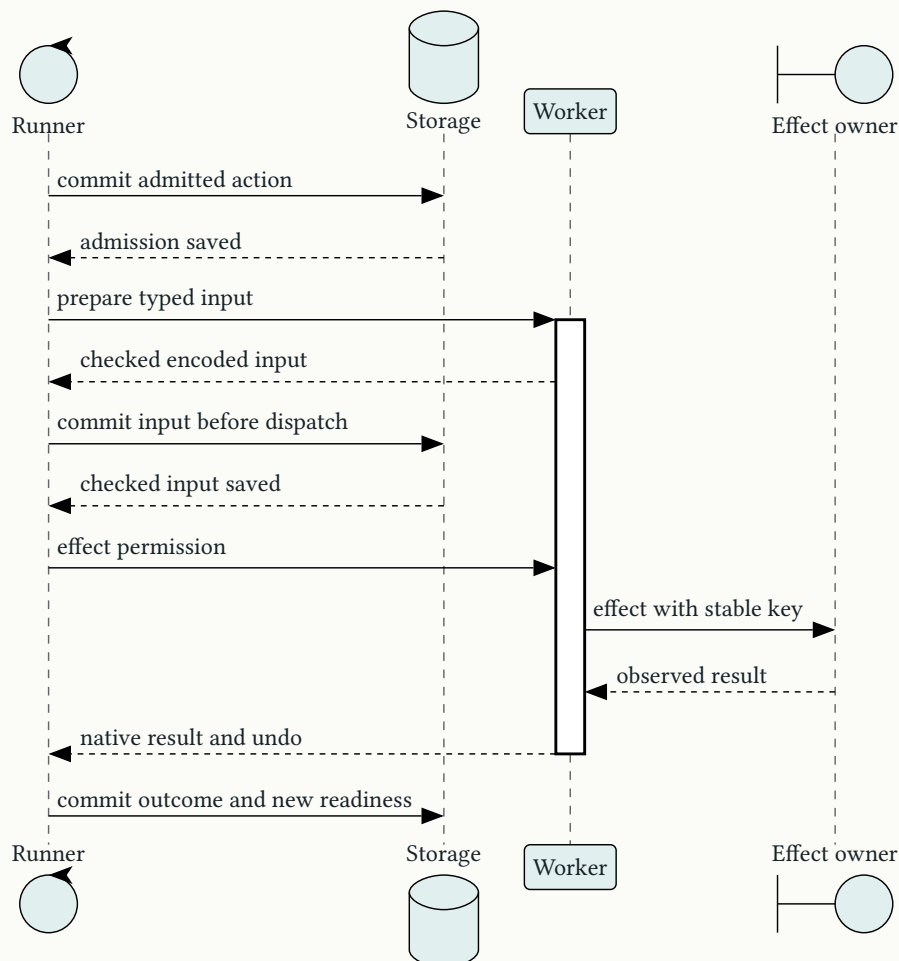
Busy, Transient, NeedsReconciliation, Incompatible, and Defect remain separate classifications; SuspensionNotSaved retains distinct original and recording failures.

- One optional persistence model and the delivery boundary are recorded in [ADR-0004](#).

1.14 Checkpoint and effect permission

Admission, checked input, and effect outcome are separate durable boundaries.

SEQUENCE Commit before effect permission



Loss after permission but before the accepted outcome creates an interrupted admitted action. The stable key and resolver must establish whether its effect happened.

Checkpoint interpreter

responsibility

Save complete execution values and reconstruct compatible run state.

interface

Opaque storage bytes; versioned envelope, typed checkpoint problems, and root/step codec boundaries.

interactions

Serializes step progress, inputs/outputs, selected decisions, retries/deadline, failure/rollback intent, journal order, settlement, and uncertainty.

invariants

Never saves executable closures, PIDs, monitors, sockets, timers, worker leases, or credentials. Every commit rewrites a complete bounded snapshot.

failure

Malformed/foreign/graph-mismatched state, insufficient restored concurrency, missing compensation input, or nonrestorable undo fails closed.

- The compatibility stamp includes workflow name/version, ordered occurrences, addresses, dependencies/choice wiring, step versions, codec versions, attempt/time budgets, and declared recovery capabilities. Runtime node ids do not enter it.
- Caller changes to callback meaning require version changes even with unchanged native types. A stamp checks declared identity, not semantic equivalence of arbitrary closures.
- Codec encoding checks that produced text decodes; it does not prove semantic equality or determinism. The codec author owns those laws and any external resources captured by callbacks.
- Undo reconstruction must be pure and may run during validation and restoration. A Continue that carries undo cannot commit if declared reconstruction returns NoUndo.
- Codec attachment, undo/resolver configuration, and error mapping preserve recovery capabilities across modifier order. A Failed resolver added after error mapping needs a compensation decider in the mapped vocabulary or fails DeciderMissingAfterMapping.
- Checkpoint format 2 retains correlation. Format 1 opens with correlation.from_key(id); the next commit uses format 2. Earlier binaries cannot read the newer envelope; no automatic arbitrary-definition migration is implied.

1.15 Recovery rules

Only explicit evidence authorizes replay of an interrupted admitted action.

Recovery resolvers

responsibility

Establish results or proven absence after interruption without repeating uncertain callbacks.

interface

Attempt and undo Evidence: Completed, Failed, NotSent, MaybeSent; compensation resolver: Some(Recovery) or None.

interactions

Receive retained native input/output and the original EffectKey; applications query downstream receipts or enforce idempotency.

invariants

Callbacks must be repeat-safe. Known saved success is reused; NotSent alone authorizes replay; MaybeSent suspends.

failure

Missing or unresolved reconciliation saves RecoveryRequired with address, action, and key. Original compensation callback is never blindly replayed.

Saved boundary	Recovery action	Constraint
Success committed	Decode and reuse	No invocation of the completed step.
Admitted before checked input	Prepare again	No effect permission was issued.
Checked input admitted without result	Attempt resolver	Completed/Failed restore result; NotSent permits original-key replay; MaybeSent suspends.
Undo admitted without result	Undo resolver	Missing/MaybeSent suspends; NotSent permits saved undo; native undo failures retained.

Saved boundary	Recovery action	Constraint
Compensation admitted without decision	Compensation resolver	Some applies original-budget decision; None suspends; never repeats original decider.
Marked uncertain returned error	Retain unknown evidence and saved decision	Uncommitted attempt uses effect resolver; interrupted decision uses compensation resolver.
Cancellation already recorded	Reconcile admitted actions before rollback	NotSent does not start a new effect during cancellation.

BEHAVIOR

A saved success is reused after runner loss

Given A step's successful output was committed before the runner stopped.

When A compatible drive resumes the execution.

Then The saved output is reused and the successful step does not execute again.

level: boundary area: Recovery

BEHAVIOR

Unknown admitted work cannot silently replay

Given An admitted action has no saved result.

When A drive resumes it.

Then Explicit resolver evidence determines its result or permits replay by proven absence.

Then Unresolved evidence suspends without repeating the external action.

level: boundary area: Recovery

- Restored max_concurrency must accommodate saved in-flight attempts and compensation decisions. A smaller setting returns ConcurrencyBelowInFlight rather than pretending those admitted actions vanished.
- A storage/codec/size/refusal failure stops workers and dispatch. If suspension recording also fails distinctly, both causes survive; otherwise the last accepted checkpoint remains authority.
- Cancellation becomes visible at the next checkpoint and may therefore include a current action's timeout. A cancellation flag changed before admission or terminal commit wins the conditional-write race.

1.16 Storage and claim lifetime

The adapter supplies atomic revocable ownership; Saga supplies workflow transitions.

Claim status unowned owned

Execution ownership entity stateful workflow owned by Storage adapter

The revocable ownership state for one saved execution.

ATTRIBUTES

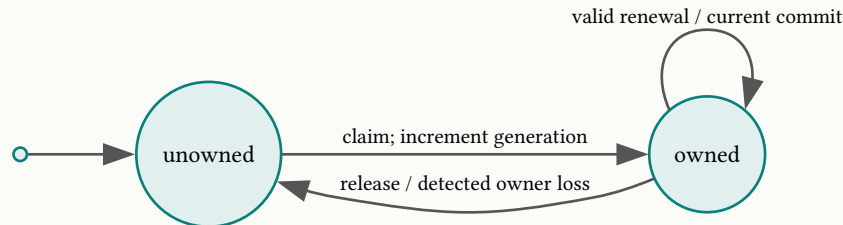
derived **Status** · Claim status [claim-status](#) · [claim-lifecycle](#)
Vacant ownership permits a new claim; adapter-supported owner-loss detection can revoke a current claim.

derived **Claim identity** · Execution id × generation × token
New claims increase generation and obtain a distinct adapter proof. A token alone does not authenticate an external business caller.

RELATIONSHIPS

1 : 1 Guards checkpoint changes of one saved execution.

STATE MACHINE Current ownership can be revoked



Lease expiry makes takeover possible under an adapter's policy. It cannot retract an external call already sent by an earlier runner; fencing guards saved writes.

Storage port

responsibility

Persist atomic execution bytes, claims, cancellation intent, and discovery.

interface

Storage.new supplies create/load/claim/commit/release/cancel/unfinished; optional renewal and operation Duration.

interactions

Returns opaque Stored and Claim values; Saga commits expected revision and observed cancellation with phase/data.

invariants

Ownership is the current claim value, not the committing process. Each refused operation leaves checkpoint bytes/revision unchanged.

failure

StaleOwner precedes CancellationChanged, which precedes Conflict; Busy excludes a competing live claim; unavailable, corrupt, timed-out, and missing records stay distinct.

Operation	Accepted effect	Refusal or idempotence
create	Revision/generation zero; uncanceled Pending bytes	AlreadyExists; never overwrites another execution.
load	Read latest bytes/revision/generation/ cancellation	NotFound or Corrupt distinct from availability.
claim	Advance generation; return Claim and Stored	Busy while live claim excludes competition.
commit	Current claim + seen cancellation + revision → new bytes/phase and revision+1	Ordered StaleOwner → CancellationChanged → Conflict.
release	Remove current ownership	StaleOwner for another claim.
cancel	Set cancellation flag without changing bytes or revision	Idempotent; durable cancel is a no-op for saved Finished.
unfinished	Up to limit unowned Pending/Suspended ids	Finished excluded; oldest first where adapter can tell.

- A heartbeat linked to the durable runner renews lease-based storage independently of checkpoint traffic. Retry backoff or a long attempt cannot depend on commit-only renewal.
- StaleOwner on renewal stops the runner. Unavailable/TimedOut renewal retries at the next interval; all later writes remain fenced if the lease expires meanwhile. A wrapper rebuilding Storage must preserve renewal.
- Drive uses a watchdog for synchronous runner storage operations; other durable operations use bounded helper calls. Timeout stops waiting and kills the helper/runner but does not itself prove whether a remote store applied an operation.
- Storage claim constructors expose adapter proofs but cannot establish validity alone. The adapter enforces current generation/token and its declared deployment scope at every mutation.

- The protocol decision is `ADR-0005`. Adapter-specific persistence and distribution guarantees remain separate from the core contract.

1.17 Storage adapters and delivery

Reference adapters differ in failure scope; delivery remains an application lifetime.

Memory storage

Keep durable records within one supervised or linked VM actor.

responsibility

Serialize the Storage protocol in one actor and monitor claiming process loss.

interface

start, supervised(name), named(name), stop, and storage(handle).

interactions

One actor serves all ids; named handles locate a restarted actor.

invariants

Current claim may be used by any process; claiming-process monitor controls owner loss. Restart discards all saved executions.

failure

Five-second operation wait returns `TimedOut`; stopped/missing actor returns `Unavailable`. Runner loss can be survived; VM/store loss cannot.

Reference file storage

Recover checkpoint bytes in a fresh VM with one directory owner.

responsibility

Apply atomic per-execution mutations using canonical-path locking and synced temporary-file replacement.

interface

Pure `file.open(canonical_absolute_directory)` returns `Storage`; caller creates and owns directory.

interactions

Erlang FFI reads/writes records; a claim records this VM's claiming process. Earlier-VM claims are unowned at recovery.

invariants

One VM at a time uses a canonical directory; lock wait is bounded. Atomic replacement preserves complete snapshots.

failure

Malformed/read/write/lock failures remain typed. Concurrent VMs, path aliases, and power-loss directory durability are outside its guarantee.

PostgreSQL persistence adapter

responsibility

Implement `Storage` across VMs with database-backed fenced claims.

interface

The separate `saga_postgres` package's public configuration/storage/migration port; caller-owned pool.

interactions

Saga owns heartbeat/drive; the adapter owns database clock, lease, schema, transactions, and generation/token checks.

invariants

No second pool is implicitly allocated by Saga; core Saga gains no database dependency.

failure

Lease loss permits recovery under the adapter's contract; external-effect uncertainty still belongs to Saga/application reconciliation.

- Detailed PostgreSQL design belongs to `integrations/saga_postgres/docs/design/design.typ`. The core owns only the persistence port and its lifetime obligations.

Delivery integration

responsibility

Wake stable executions and map delivery interruption into explicit Saga operations.

interface

Application-owned stable id, reconnect/read/drive/cancel, unfinished scanning, and `error_kind` mapping.

interactions

A Grind job supplies capacity and wakeup; Saga's saved outcome supplies workflow completion authority. Fabric supplies parent policy and continuation.

invariants

Driver stop is resumable interruption; cancel is recorded intent. No automatic full-local-workflow retry follows redelivery.

failure

Busy/Transient can be retried deliberately; NeedsReconciliation needs operator/application evidence; Incompatible/Defect require correction. Atomic job admission needs an application transaction/outbox.

- The checkout application selects Grind cancellation while driving Saga and explicitly calls durable.cancel when cancellation is intended. Stopping drive alone must leave the execution recoverable.
- Delivery deadlines reserve time for Saga stop, drain, release, settlement, and any persistence owner-loss window. Retrying a completed failed workflow is an explicit application policy.
- An application using a shared pool starts migrations and stops infrastructure under its own lifecycle. Store reuse does not merge job, agent, and workflow claim authority.

1.18 Observation and verification ports

Observations describe transitions; executable consumers test guarantees at their actual boundary.

Telemetry

responsibility

Emit package-owned lifecycle facts after corresponding transitions.

interface

Six typed event descriptors: run_started, run_stopped, step_started, step_stopped, compensation_stopped, undo_stopped.

interactions

Sinal transports typed measurements/metadata; step clients read the same correlation from EffectKey.

invariants

Every event carries correlation; durable metadata includes execution id; no telemetry value drives workflow state.

failure

Handler raises are isolated by Sinal. Synchronous slow handlers delay coordinator processing and therefore timing; no guaranteed durable observation delivery is implied.

- The local correlation defaults to unique(); the first durable drive saves an explicit handle correlation or from_key(id). Later handles cannot change it; [ADR-0006](#) records the identity contract.
- Saga emits no separate claim/release/lease/checkpoint event family. Applications obtain those facts from durable/storage results rather than interpreting a missing event as absence of an effect.

Testing and conformance

responsibility

Expose observation and extension checks without private imports.

interface

testing.wait_until polls public Progress with an explicit timeout; storage.conformance.run accepts fresh fixtures and separate owner-loss allowance.

interactions

Consumer tests supply native records/errors and public storage/resolver fakes; fresh VM probes exercise file recovery.

invariants

Conformance checks atomic create, claims-as-values, live/lost ownership, stale writes, cancellation race, listing, release, and killed-runner recovery.

failure

Located suite failures report the violated contract. A one-VM pass proves neither distributed fencing nor media durability; adapters need separate deployment tests.

Evidence	Guarantee checked	Boundary
examples/order_consumer	Common/advanced native use, mapped errors, failures/undo, durable recovery, Blueprint bridge, full reporting	Separate package using public modules only.
fixtures/negative + positive controls	Opaque Port/EffectKey/Execution and wrong error/choice/Continue types	Actual compiler failures with expected diagnostics.
test scheduling/lifecycle/rollback/retry/unknown/reporting	Admission, timers, owner loss, reverse undo, evidence and mailbox cleanup	Public behavioral and process probes.
scripts/check_durable_restart.sh	Concurrent effect/compensation receipts and stable-key reconciliation	Actual VM kill and fresh definition in another VM.
oracle + PROVENANCE	Seven scoped Reactor scenarios; reverse undo and sibling settlement are deliberate differences	Reactor 1.0.6; not full Reactor API parity.
bench	Build-once/store/readiness cost and correctness checks	Measured evidence; no universal latency guarantee.

- Operational commands remain in README, AGENTS, DURABILITY, and oracle provenance. The design gate checks notation, links, and PDF freshness; it does not replace runtime, fault, compiler, or adapter evidence.

1.19 Retained composition contracts

Expanded capabilities retain explicit ownership instead of changing the static facade's meaning.

1.19.1 Value-dependent traversal and local continuation

Typed expansion

responsibility

Compose known native types after a value arrives and traverse a homogeneous runtime collection.

interface

A future value-dependent composition/traversal surface over the canonical Workflow; no accepted replacement signatures.

interactions

Expands typed local fragments; durable fragments need stable factory identity/version and saved branch selection.

invariants

Ordering, graph-size, cancellation, and shared/partitioned capacity must be explicit. Arbitrary heterogeneous steps and named native-result lookup remain excluded.

failure

Unsupported durable local closures are refused at eligibility; runtime collection/resource bounds must fail explicitly before uncontrolled expansion.

- Explicit local halt/resume retains typed state and control authority. Copyable execution values must not permit repeated external effects merely because a completion or wake token matches.
- A future paused or blocked recovery retains native input, original cause, continuation, and completed journal. Retry permission, cancellation/rollback permission, and reconciliation authority remain separate; relative wake tokens reject stale or duplicate delivery. The delivered Hold is a terminal report, not this resumable recovery surface.

1.19.2 Approvals and historical review

Approval continuation

responsibility

Suspend at an authenticated answer boundary without treating waiting as business failure.

interface

Retained input, wait identity/revision, authenticated answer, command identity, deadline, and explicit continuation owner.

interactions

Commits waiting state and releases worker capacity; Fabric owns agent approval policy while Saga owns its workflow boundary.

invariants

Stale/duplicate signals cannot resume another revision. Forking from history cannot silently reuse compensation authority over prior effects.

failure

Expired, rejected, cancelled, stale, incompatible, corrupt, and ambiguous-effect outcomes stay distinct. Prior-effect policy for forks remains a ruling.

1.19.3 Independent children and compensation authority

Child boundary

responsibility

Attach an independently owned execution without flattening its journal into its parent.

interface

Typed child contract with actual input/output/error/undo-error codecs; prepared command; checked admission receipt; observation and cancellation intent.

interactions

Parent persists route, exact request, stable command, and attachment; after restart it reattaches to compatible child owner state.

invariants

Compatibility and authority precede decoding. Waiting releases capacity needed by child; parent and child recovery remain distinct.

failure

Unknown/invalid admission preserves exact request for reconciliation; later failed retry does not resolve earlier uncertainty. Missing owner/run, pruned output, incompatibility, and corrupt result differ.

Compensation authority

responsibility

Authorize one logical compensation operation for a retained successful execution.

interface

Owner-backed exclusive claim, durable receipt reference, typed rebound undo-error codec, stable request command, and separate compensation observation.

interactions

Owner checks success, journal retention, claimant identity, and current claim; parent retains attachment and receipt references.

invariants

Success alone grants no authority. Copyable handles never enforce single use; storage admits at most one logical operation per claim and replays its original admission.

failure

Retention expiry/pruning, competing claim, unauthorized caller, unavailable owner, incompatible contract, uncertain admission, and undo failure remain distinct.

Independent state axis	States	Rule
Historical child result	Pending / completed / failed / unresolved; output may be pruned	Compensation does not rewrite historical success.
Compensation operation	Pending / running / succeeded / failed / unresolved	One logical admission per authority claim.
Result and journal retention	Available / expired / pruned	A retained journal may outlive output; replayed admitted request retains identity after fresh-admission expiry.

- The laboratory establishes these distinctions through scripted immutable owner records. Concurrent storage enforcement, authentication, actual compensation, restart, and local post-success authority remain the pending implementation boundary.

1.20 Runtime-authored graphs

Finite data contracts type runtime wiring without pretending to create new native Gleam types.

Runtime graph compiler

responsibility

Turn an editable graph into a checked immutable executable definition.

interface

Versioned node registry, parametric port expressions, configuration contracts, draft graph, publication errors, and PublishedGraph.

interactions

Blueprint owns finite runtime contracts/validated values; Saga owns registration, instantiation, wiring, graph validation, and execution.

invariants

Only PublishedGraph executes. Runtime data remains opaque validated values; static adapters retain actual native codecs.

failure

Unsupported schemas, unresolved versions/types, invalid configuration, illegal connections, cycles, missing inputs, and resource limits are located publication failures.

Runtime graph status

draft

published

retired

Runtime graph definition

aggregate

stateful

workflow

owned by Saga runtime graph compiler

The retained runtime-authoring model with editable and immutable states.

ATTRIBUTES

derived

Status · Runtime graph status [runtime-graph-status](#) · [runtime-graph-lifecycle](#)
Publication derives executable capability; retirement and existing-run retention require an explicit policy before implementation.

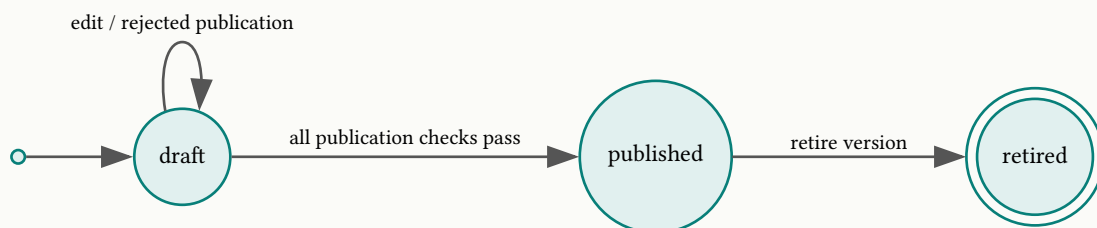
authored

Node contract references · Registry kind × version × finite port contracts
Deployed implementations remain separate from serializable graph data.

derived

Resolved schema bindings · Concrete finite runtime contracts
Publication resolves every variable and validates values/connections before execution.

STATE MACHINE Runtime graph publication



A changed executable graph is a new version derived from a draft. Treatment of active executions and registry retention on retirement remains explicit design work.

1.20.1 Parametric node model

Family	Type relation
Identity / Delay	$T \rightarrow T$
Choose	$\text{Bool} \times T \times T \rightarrow T$
Collect / Constant / Validate	$T... \rightarrow \text{Array}(T); () \rightarrow T; \text{wire value} \rightarrow \text{validated } T$
Map	$\text{Array}(A) \times \text{Graph}(A, B) \rightarrow \text{Array}(B)$
LLM	Input $\rightarrow$ Output under declared finite contracts

- A node definition retains kind/version, input/output port definitions, configuration schema, and deployed implementation. Type expressions include concrete contract, variable, array, optional, and finite object.
- Instantiation binds variables through incoming connections, including array/optional/object structure. The first compatibility rule is exact structural equality with explicit transformation nodes; width subtyping and broader assignability require separate acceptance.
- Every runtime port has an opaque identity and validated contract; every node output is validated against its output contract. Required field access derives the field contract and returns a located failure.
- A native adapter validates and decodes input before its native callback, then checks encoding and validates output. A custom codec with unavailable schema cannot silently claim compatibility.
- The LLM node composes llm_wire provider execution and Blueprint output validation. Fabric can supply bounded agent policy; it does not become Saga's provider transport.

Runtime node execution

responsibility

Execute a published node through its checked concrete port contracts.

interface

Opaque port identifiers and maps of Blueprint validated values; concrete input/output contracts and located node errors. Native adapters decode typed input, invoke their original handler, encode output, and validate the encoded result.

interactions

Saga schedules the registered implementation; Blueprint checks contract values; application services own actual remote effects and reconciliation.

invariants

A successful node output conforms to its declared contract. Registry identity and publication remain fixed for one execution; a dictionary never permits unchecked native-value lookup.

failure

Input decoding, native handler failure, output encoding, invalid node output, timeout, and uncertain remote effects remain distinct. Publication does not prove external provider deduplication.

1.20.2 Publication and hostile input

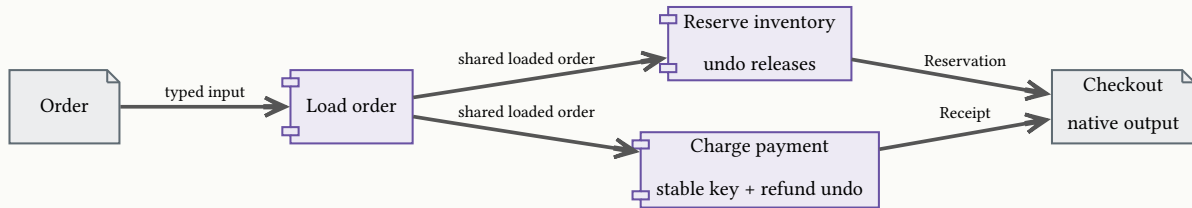
Publication stage	Required acceptance
Configuration and registry	Decode every configuration; resolve definition/version; validate constants and defaults.
Type derivation	Instantiate parameters; unify connected ports; derive output schemas.
Graph structure	Detect cycles; reject missing producers and undeclared duplicate writers; require associative declared reducers.
Boundary completeness	Check required inputs; validate graph input/output contracts; reject unreachable outputs.
Executable publication	Produce one immutable PublishedGraph with all type variables resolved.

- Finite contracts forbid cyclic references for this surface; arrays and nested objects still remain finite schema trees. Schema depth/size, graph nodes/edges, traversal, and execution resource budgets need explicit bounds before hostile-input acceptance.
- Serialization saves graph data and versioned implementation identity, never closure code. Restore checks registry versions and schema/value compatibility before execution.
- Visual editors, diagnostic metadata, exported graphs, hostile publication tests, and persistence/recovery scenarios belong to this retained surface; static Port compiler tests do not verify it.

1.21 End-to-end walkthrough

An order reserves inventory and charges payment while preserving uncertainty and cancellation authority.

VIEWPOINT DATAFLOW · ALTITUDE L3 · COMPONENTS One typed order workflow



LEGEND


value object
attribute-defined value


component
owned runtime part


dataflow
weighted directed link

The external order consumer supplies native *Order*, *Reservation*, *Receipt*, *PayError*, and *UndoError*. *Load* runs once while independent branches run within the selected capacity.

- Define the graph once at application startup. Pure configuration sets concurrency and distinct attempt/deadline/settle/cleanup bounds before one local run starts.
- A clean charge and reservation produce native *Checkout*. A known payment decline fails with the typed business error; the inventory undo succeeds or contributes its located native *UndoError*.
- A *MaybeCharged* error marked by *unknown_when* can request a keyed retry. Even if retry succeeds, the earlier unknown attempt remains visible in *CompletedWithUnknownEffects*. Without a settling decision, default *Hold* retains inventory for reconciliation.
- For persistence, attach versioned root/step codecs and effect/undo/compensation resolvers to the same graph. *start_or_reconnect* stores one id/input before drive; a repeated compatible request reuses that execution.
- If the runner dies after the provider receipt but before checkpoint outcome, the next compatible drive invokes the resolver with the original key. *Completed* restores the receipt; *NotSent* proves replay permission; *MaybeSent* suspends.
- Explicit durable cancellation records intent, reconciles admitted work, and undoes only known completions. Driver/job death alone preserves resumable state. A survivor reads the saved full report or receives local owned-report settlement.
- The application retains the typed *Outcome* for audit/recovery and chooses *outcome.classify* for public behavior. Safe summary keeps settlement facts without exposing payloads; reporting operational error stays separate from obtained workflow failure.

Glossary

Every term the layer declares, with the context that owns it.

Approval

owned by root

A suspended decision boundary retaining input and requiring an authenticated, revision-checked, deduplicated answer.

Attempt

owned by root

One invocation of a step under its positive attempt number and fixed attempt budget.

Checkpoint

owned by root

A complete saved data representation of durable execution progress and its recovery authority.

Child

owned by root

An independently owned workflow execution attached to a parent through an explicit admission and observation contract.

Claim

owned by root

The revocable execution ownership value identified by execution id, increasing generation, and adapter token. *Avoid:* process identity — a current claim can be used by another process.

Codec

owned by root

A versioned pair of fallible conversions between a native value and saved text.

Compatibility stamp

owned by root

A computed identity of the workflow structure, semantic versions, codecs, and declared recovery capabilities used to admit saved state.

Compensation

owned by root

The failed-attempt decision that selects retry, replacement output, abort, or held uncertainty. *Avoid:* undo — undo addresses a known completed step.

Compensation authority

owned by root

The explicit owner-backed right to admit compensation of a successful child or completed execution.

Correlation

owned by root

The Sinal identity carried by an execution's action contexts and observations for joining application evidence.

Drive

owned by root

One bounded runner attempt to advance a durable execution until it finishes, suspends, loses ownership, or exhausts its execution budget.

Durable execution

owned by root

One saved execution identified within a storage and recoverable through a compatible workflow definition.

Effect key

owned by root

An opaque action context containing stable idempotency identity, attempt identity, attempt number, and execution correlation.

Execution

owned by root

One interpretation of a workflow for one input, with its own action progress, completed values, journal, and terminal evidence.

Execution handle

owned by root

An opaque reference to a running local coordinator and its outcome-delivery ownership.

Hold

owned by root

A compensation decision retaining application evidence without authorizing retry or rollback of completed steps.

Local run

owned by root

One execution of a workflow whose lifetime belongs to the process that starts it.

Outcome

owned by root

The typed terminal report of completion, completion with unknown effects, failure, cancellation, or unresolved held evidence.

Persistence capability

owned by root

The checked association of an existing workflow with root codecs, compatibility identity, and checkpoint limits.

Port

owned by root

An opaque typed reference to a dependency value within one workflow construction. *Avoid:* result name — names identify steps but do not transport native values.

Published graph

owned by root

An immutable executable runtime graph with resolved node versions, type parameters, configurations, and checked connections.

Report receiver

owned by root

The independent process retaining outcome-delivery responsibility for one short-lived invocation owner.

Resolver

owned by root

An application callback that establishes the result or absence of an interrupted admitted action without blindly repeating that action.

Revision

owned by root

The increasing count of committed checkpoint changes for one durable execution.

Run handle

owned by root

An opaque durable reference combining a persistence capability, storage, execution identifier, and drive configuration.

Runtime graph

owned by root

A graph authored after native compilation whose values and connections are checked against finite runtime data contracts.

Settlement

owned by root

The complete retained evidence about undo, remaining known effects, interrupted actions, sibling failures, and unknown effects after admission stops.

Step

owned by root

A typed operation definition with its attempt, recovery, undo, timing, and optional persistence capabilities.

Step address

owned by root

The structural identity of a step occurrence: nested scope, name, and positive occurrence rank within that scope and name.

Step occurrence

owned by root

One structurally addressed use of a reusable step inside a workflow definition and execution.

Storage

owned by root

The atomic persistence port for saved bytes, execution claims, conditional commits, cancellation intent, and unfinished-execution discovery.

Undo

owned by root

An application action intended to reverse a known completed step using its retained input, output, and own effect key.

Undo journal

owned by root

The ordered completed-step records that retain undo capability and actual completion order.

Unknown effect

owned by root

An action whose effect status is unresolved because it crashed, timed out, was interrupted, or returned an application error classified as uncertain.

Workflow

owned by root

A reusable typed definition of dependency-connected steps with one input, output, business-error, and undo-error contract.